Web-Based Stock Scoring Engine

Leul Ejigu

April 2, 2026

1 Introduction

Retail investors today have access to more financial data than ever before, but access does not necessarily guarantee clarity. In theory, anyone can access earnings reports, valuation ratios, analyst headlines, and market data for the entire S&P 500. In reality, the high volume of information and inconsistency can make decision-making difficult, especially for beginners. My Junior IS, Web-Based Stock Scoring Engine, addresses this gap by turning these financial inputs into a ranked list of companies. My idea is simple: instead of asking users to manually compare hundreds of data points across hundreds of firms, the system calculates a standardized score based on thirteen metrics, which I believe indicate a good stock, and presents the results in a user-friendly manner. You may be wondering about the accuracy of these metrics. However, I believe that determining the accuracy of these 13 metrics would make a different topic, and it is more economics focused.

This project matters because most investor-facing tools either over-simplify or over-complicate. Some investor-facing tools just show a few key stats and leave out a lot of important details. And others present extensive tables and charts but leave the synthesis entirely to the user. My approach is intentionally in the middle: automate the heavy computation then present both summary and drill-down views so users can quickly identify strong candidates and still inspect why each candidate ranked where it did. This design approach argues that financial interfaces should separate and organize information based on decision value, rather than only maximizing the amount of data displayed on screen. [5]. The project is being built as a web-based system with a Python backend and API layer. The current setup includes key ingestion workflows for S&P 500 companies and financial statement data with workflows built using the pandas library. In practical terms, the workflow consists of four stages: first, acquire a clean snapshot of the S&P 500 universe. Second, ingest raw fundamental and price data. Third, normalize these fields into metric components. And lastly aggregate them into a final score for ranking.

A major part of this Junior IS is going to consist of the integration of both quantitative and qualitative evidence into one ranking framework. Quantitative metrics (market cap, P/E Ratio, operating margin, short interest . . .) are straightforward to compute from structured data. Qualitative metrics (leadership stability, competitive moat, news-driven catalysts . . .) are harder because they depend on unstructured text and context. Rather than ignoring these qualitative signals, I plan to utilize a large language model (LLM) API to evaluate

qualitative signals and aim to convert these signals into comparable scores [8, 4]. This is important because many real investment decisions are not made from financial ratios alone.

Another contribution is interface design, even a statistically sound model can fail users if the output is hard to read. Lately, there's been a lot of focus in financial big-data visualization on making the results of models easier to understand. After a model prints out some results, the way we show that information should really highlight trends and summaries that users can actually act on. [3]. My system adopts that principle by prioritizing top-ranked summaries while preserving detailed metric breakdowns on demand. This project is not attempting to produce guaranteed price prediction or claim market-beating returns. The objective is to provide decision support rather than to make precise forecasts. Users should be able to see what data was used, what assumptions were made and how each component influenced rank position. [1].

Overall, this Junior IS presents the stock scoring engine as a link between the high amount of financial data and the quality of investor decisions. It offers a ranking system that's easy to follow, a scoring model that mixes different methods and a user-friendly approach to handling lots of market information.

2 Background

To understand the design, it is important to define the main concepts and constraints. At first, the plan was to rank all the stocks on the market, but because it was hard to find financial data for all of them, i had to narrow it down. So decided for the data to be based on the S&P 500, which is not just a random list of large U.S. companies. The index has clear rules from S&P Dow Jones Indices about which companies can be included and kept in it like liquidity and overall market structure.[10]. For a ranking web app, this matters because the results are only as good as the stocks being ranked. If the list includes stocks that should not be there, outdated ones or ones that are barely traded, the rankings can be misleading. That's why my project starts with a clear list of the actual constituents before any scoring happens.

Second, the project is built around working with data in tables. In Python, pandas library is the core tool because it makes it easy to work with labeled columns, handle missing values and merge data from different sources. [6].In practical terms,it lets our program combine company info, financial statement data, and price data into one clean table that's easy to compare.

Third, the web app looks at more than one thing. A stock can be cheap but not growing much or profitable but still risky financially. Since those factors are measured differently, I have to standardize them and give them weights before putting them together. That's why I use a multi-criteria approach, because it helps combine different strengths and weaknesses into one overall ranking.[1]. In my project, this means each metric adds to the final score without letting one single factor control the whole ranking.

Fourth, the system recognizes between hard and soft information. Hard information includes numeric and structured fields such as revenue, net income, debt, liquidity ... Soft information includes narrative and contextual signals such as management quality, strategic direction, and major news catalysts. Research I have read argues these categories should

be surfaced differently because users process them differently during decision-making [5]. My interface is built around that idea: hard metrics are there for direct comparison in the ranking table, while soft metrics are still shown but in a more supportive way, so they add context without making the main view feel crowded.

Fifth, working with qualitative data brings in NLP challenges. Financial news and text can be specific, fast-changing, and sometimes hard to interpret. Earlier systems like AZFin showed that pulling useful structure out of news can help make better market judgments.[8]. Later work found that character-level models handle specialized finance vocabulary and brand-new terms better than word-based models.[4]. For this project, these findings support my decision to use an LLM API to help process selected qualitative parts instead of relying on simple keyword counts.

Sixth, deployment matters because even if the ranking system works well in theory, it won't be very useful if updates are slow. [9]Since my current version is still early and not fully serverless yet, I'm not treating scalability as the main focus right now. But this research gives me a solid direction for how I could scale it later if the project ever grows beyond a class prototype.

Finally, UI/UX is not just about making the app look good. In a financial app, it actually affects how correctly people understand the data. Even if the numbers are right, a confusing design can still lead to bad conclusions. That's why things like readability, keeping the screen from feeling overwhelming and only showing extra details when needed are important for making the tool useful and trustworthy [7]. That perspective aligns with my design objective: a ranked table for quick understanding, plus deeper per-stock metric views for verification and user confidence. Together, these ideas make up the main focus of my Junior IS: choosing the right stock universe, building a reliable data workflow, combining different metrics into one ranking, using both quantitative and qualitative data, thinking about scalability, and designing the interface in a way that helps users make better decisions.

3 Related Works

The literature relevant to this project can be grouped into four themes: 1. financial information presentation and visualization, 2. quantitative scoring and ranking methods, 3. textual/AI approaches for qualitative data and 4. system architecture for real-time. Looking at these themes together helps show both what is already known and where my thesis adds something new. In terms of how financial information is presented, Ettredge, Richardson, and Scholz give a basic framework for how investment-focused websites should organize information[5]. Their distinction between hard and soft content remains directly useful for modern dashboards. The biggest strength of that work is how clear the idea is: it shows that just adding more numbers doesn't automatically help people make better decisions. The main limitation is that it's older, so it doesn't reflect how modern financial tools work today, for example, using AI to score qualitative info. My project builds on their idea by applying the hard vs. soft metric. More recent work by Dong, Huang, and Wang looks at financial big-data visualization from a machine learning perspective.[3]. What stands out most from their framework is the idea of looking at visualization before, during, and after modeling. The after-model stage matters the most for my project, since my system focuses

on presenting the final rankings in a way users can understand without feeling overloaded. Compared to Ettredge et al., Dong et al. pay more attention to large-scale data and model-based workflows. But their work is also broader and not really focused on stock-ranking dashboards. My project takes that post-model idea and applies it more directly to a ranking UI built around the S&P 500.

For quantitative scoring and ranking, MCDM literature gives the strongest method behind my approach. Behzadian et al.'s survey of VIKOR methods explains how different criteria that may conflict with each other can be adjusted and combined into one balanced ranking.[1]. This connects directly to the main challenge in my 13-metric model, which is combining different types of indicators without letting one metric overpower the rest just because it uses a bigger scale. The main point is not that I need to copy one specific algorithm, but that I need solid adjusting and weighting to make the final score fair and reliable. One thing the VIKOR literature does not really focus on is how to apply these ideas in a web-based financial dashboard that also includes qualitative data. My project helps fill that gap.

The bibliography also includes more recent work on improving UI/UX in financial software at the application level. [7]. This kind of work supports the idea that interface quality affects more than just how users feel about the app, it can also affect how they understand the results and the decisions they make. When combined with the older financial web-design model, it helps make a clear point. One limitation is that a lot of UX research stays pretty general and does not explain exactly how interface design should connect to the scoring system underneath. My project adds to that by linking UI choices directly to the ranking model, like showing an overall rank while also giving a clear breakdown of the individual metrics behind it.

For qualitative and text-based market signals, Schumaker and Chen's AZFin system is an important early example.[8]. It demonstrates that unstructured financial news can be transformed into predictive features through systematic text analysis. For my Junior IS, the main point is that qualitative signals don't have to stay as just opinions , they can be turned into something the system can measure and use. At the same time, AZFin comes from an older era of NLP, so it had tighter limits on how flexible the models were and what kinds of language they could handle compared to today's transformer-based systems. Building on that idea, the character-level neural approach by dos Santos Pinheiro and Dras goes a step further, showing that these models can handle finance-specific wording and even unfamiliar text patterns better than more word-based approaches. [4]. Still, neither paper fully solves the deployment and interpretability issues that come up in real dashboards. My project borrows from both to support AI-assisted soft metrics, while still keeping things transparent by showing the component scores behind the final ranking.

On system architecture, Solaimalai's serverless analytics paper shows a practical way to scale stock data collection and processing when demand goes up. [9]. This is valuable for the long-term roadmap of my project, which may need frequent updates across hundreds of symbols. The clear advantage of this literature is operational focus: latency, concurrency, and cost behavior are treated as first-class design concerns. The current limitation for my thesis stage is implementation scope. My prototype is currently pipeline- and API-centered rather than fully serverless. Nevertheless, the paper provides a credible architecture direction for future extensions. Finally, the S&P methodology document and pandas foundation paper

play a different but essential role. [10, 6].The main strength of this work is that it focuses on the practical side of the system, like speed, handling multiple requests, and cost. For my project, the limitation is that my current version is still a prototype, so it is more focused on the system and API than on being fully serverless. Still, the paper gives me a solid direction for how the system could be built out later. Across all of these themes, one clear pattern shows up. A lot of existing research looks at important parts of the problem, but usually one piece at a time. Some focus on interface design without modern AI, some focus on ranking methods without building a full product, some look at NLP without explaining how results should be shown clearly in a dashboard, and others talk about system architecture without combining hard and soft scoring. The gap is a system that brings all of this together in a way that is solid technically, easy for users to understand, and realistic to build at the S&P 500 level. That is where my thesis fits in. It combines reliable data collection, multi-criteria scoring, qualitative signal analysis, and decision-focused visualization into one web-based ranking system.

4 Methodology

This project is about building a stock ranking system for companies in the S&P 500 that is easy to understand. The idea is to rank companies using both financial data and qualitative information, such as CEO-related context, while also making it clear to the user why a company ended up where it did in the ranking. I decided to work only on the S&P 500 because it gives a more reliable and realistic group of companies to work with. These firms are large, well-established, and usually have better reporting transparency and liquidity than smaller public companies [10]. This project starts by collecting the list of S&P 500 companies, then pulls financial and market data for each one (revenue, net income, debt, operating income, and related indicators). After that, the data is cleaned and organized using pandas so missing values or inconsistent entries do not affect the results [6]. Once that is done, the system combines them into a weighted score based on thirteen different metrics to create the final ranking. On top of that, the project also includes qualitative signals using LLM-assisted analysis. In the end, the goal is to produce a working web-based stock scoring engine where users can see the rankings, explore the scores, and understand the reasoning behind them.

This project does not rely only on financial ratios. It also includes qualitative information so the rankings provide a more complete picture of a company. Financial statements can show how a company has performed in the past, but they often miss factors such as leadership strength or whether the company has a clear plan for the future. To address that gap, the system uses an LLM to analyze less structured information, including company context and executive leadership profiles. For example, it can evaluate how a CEO is viewed and whether that leadership appears stable or risky. The LLM reads selected text through an API, scores it in areas such as market sentiment, strategic clarity, and management quality, and then converts those ratings into numeric components used in ranking [8, 4]. On the financial side, the metrics also need to be standardized because measures such as operating margin and short interest are expressed on different scales. Without normalization, one metric could dominate the final score simply because its values are larger. To prevent that, each metric is

normalized before final scoring, and missing values are removed. After that, both financial and qualitative scores are combined into the final score and shown in a web-based interface that allows users to compare companies and understand how the rankings were determined.

4.1 System Scope and Research Goal

In this subsection, I will define the main goal of the project: building a decision-support tool for S&P 500 stocks rather then a guaranteed price-prediction model. I will explain why the system focuses on ranking companies according to a weighted set of metrics and why that framing is appropriate for this Junior IS.

4.2 Data Collection and Preparation

In this subsection, I will describe how the system builds a clean stock universe and gathers the data needed for scoring. This includes obtaining the current S&P 500 constituent list, importing company, price, and financial statement data, and cleaning the resulting tables with Python and pandas so the later scoring steps use comparible inputs.

4.3 Metric Design, Normalization, and Weighting

In this subsection, I will describe the thirteen metrics used in the model and explain how they are transformed into comparible component scores. I will also explain how normalization and weighting prevent any one raw measurement from dominating the final ranking simply because it has a different scale.

4.4 Qualitative Signal Scoring

In this subsection, I will explain how the project handles non-numeric information such as leadership stability, competitive moat, and recent catalysts. I will describe how an LLM API is used to evaluate selected qualitative evidence and convert it into a score that can be combined with the structured metrics.

4.5 Ranking Pipeline and Interface Output

In this subsection, I will explain how the component scores are aggregated into a final ranking and then presented to the user. I will describe the ranked summary view, the per-stock drill-down view, and the design choice to show both an overall score and the metric breakdown so users can verify why a company appears where it does.

4.6 Reproducibility

In this subsection, I describe the high-level workflow and components needed so the project can be reproduced consistently.

5 Implementation

The stock scoring engine is built as a web-based application with a Python backend and a browser-based frontend. Python was selected because it is effective for data handling and integrates well with financial libraries, APIs, and LLM workflows [6]. Most of the processing happens in the backend, where the system follows a pipeline from data collection to final score output. It starts by retrieving the list of S&P 500 companies (via Wikipedia scraping), then pulls the financial and market data required for the model from external APIs (FMP API). After collection, the data is cleaned because companies do not always report values in the same format and some fields can be missing. Pandas is used to structure the tables and handle missing values [6]. Once cleaned, the data is stored in a structured format such as CSV so the system does not repeatedly call the same APIs, which improves runtime efficiency. The metrics are then normalized, weighted scores are calculated, and final rankings are sent to the frontend. On the frontend, users can view ranked companies and inspect score breakdowns to understand why each company received its position.

6 Results

This section is not meant to prove that the system can beat the stock market. The point is to show that the engine can actually do what it was designed to do. In this project, a good result means the system can take the list of S&P 500 companies, collect the needed data, calculate the scores, and present the rankings in a way that users can understand. The main output is a ranked table of companies based on their overall scores. That table shows the ticker, company name, overall score, and the breakdown of the thirteen metrics used in the model. This matters because users should be able to look beyond the final ranking and understand what pushed a company higher or lower. For example, one company might score well in profitability but poorly in qualitative areas, while another may be more balanced across the board, and those differences should be visible in the final breakdown.

6.1 Ranking Output Examples

In this subsection, I will show example rankings generated by the current version of the engine. These examples will demonstrate what the final output looks like and how the system orders companies once all component scores have been combined.

6.2 Tables, Figures, and Metric Breakdowns

In this subsection, I will identify the tables and figures that will appear in the final paper. These will likely include a sample ranked table, a per-company metric breakdown, and a figure or diagram showing the scoring pipeline from raw data to final rank.

6.3 Evaluation Criteria

In this subsection, I will explain what counts as success or failure for this project. Success will mean that the system can consistently ingest data, compute scores without obvious

breakdowns, and present rankings in a way that is understandable and inspectable by a user.

6.4 Preliminary Limitations of the Results

In this subsection, I will acknowledge the limitations of the results I can report at this stage of the project. For example, I may have prototype-level outputs rather than a full-scale deployment, and I may not yet have a large user study or long-term market validation.

7 Conclusion

In this section, I will summarize the main contribution of the project and restate why a stock-scoring engine can help users manage financial information more effectively. I will also close by discussing the current limitations of the prototype and the most important next steps for extending the system.

7.1 Main Contribution

In this subsection, I will restate the core contribution of the Junior IS: combining structured financial metrics and AI-assisted qualitative signals into one web-based ranking workflow. I will emphasize that the project is a decision-support system designed to improve clarity, not a system that guarantees market-beating predictions.

7.2 Current Limitations

In this subsection, I will acknowledge the main weaknesses of the current prototype, including the limited project scope, the challenge of validating subjective qualitative scores, and the fact that the system is not yet build for full-scale production deployment.

7.3 Future Work

In this subsection, I will explain how the project could be extended in the future. Likely next steps include improving the evaluation strategy, scaling the architecture, refining the qualitative scoring process, and adding more polished visualizations or user testing.

References

[1] Behzadian, M., Otaghsara, S. K., Yazdani, M., and Ignatius, J. A state-of-the-art survey of vikor methodology. *International Journal of Information Technology & Decision Making 11*, 01 (2012), 139–181.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, Cambridge, MA, 2009.

[3] Dong, X., Huang, W., and Wang, J. Financial big data visualization: A machine learning perspective. In *Proceedings of the 17th International Symposium on Visual Information Communication and Interaction (VINCI '24)* (New York, NY, USA, 2024), ACM. Accessed: 2026-01-27.

[4] dos Santos Pinheiro, L., and Dras, M. Stock market prediction with deep learning: A character-based neural language model for event-based trading. In *Proceedings of the Australasian Language Technology Association Workshop 2017* (2017), pp. 6–15. Accessed: 2026-02-19.

[5] Ettredge, M., Richardson, V. J., and Scholz, S. A web site design model for financial information. *Communications of the ACM 44*, 11 (2001), 51–55. Accessed: 2026-01-27.

[6] McKinney, W. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (2010), S. van der Walt and J. Millman, Eds., pp. 56–61. Accessed: 2026-01-27.

[7] Runsewe, O., Akwawa, L. A., Folorunsho, S. O., and Osundare, O. S. Optimizing user interface and user experience in financial applications: A review of techniques and technologies. *World Journal of Advanced Research and Reviews 23*, 03 (2024), 934–942.

[8] Schumaker, R. P., and Chen, H. Textual analysis of stock market prediction using breaking financial news: The azfin text system. *ACM Transactions on Information Systems 27*, 2 (2009), 12:1–12:19. Accessed: 2026-01-27.

[9] Solaimalai, G. Serverless architecture for real-time stock market data analytics in cloud environments. *International Journal of Computer Applications 186*, 75 (2025), 9–15.

[10] S&P Dow Jones Indices. S&p u.s. indices methodology. Tech. rep., McGraw Hill Financial, February 2016. Accessed: 2026-01-27.